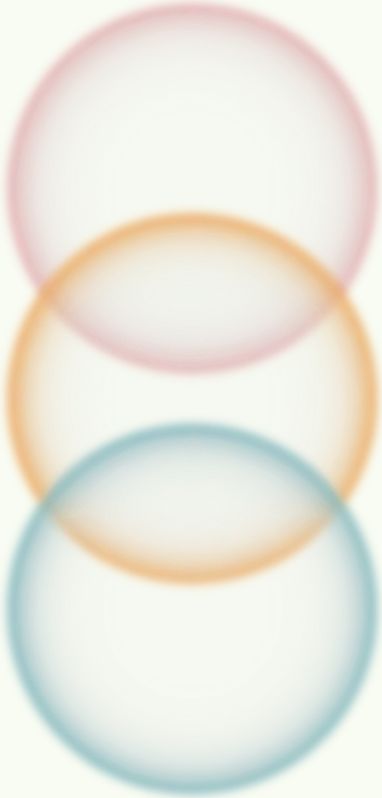
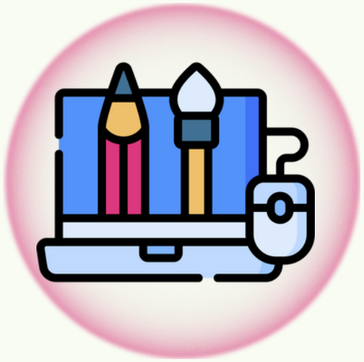
Aspectos metodológicos del diseño

Instructor Manuel Porras
mf.porras@uniandes.edu.co



Vamos a ver unas recomendaciones y consejos para el diseño

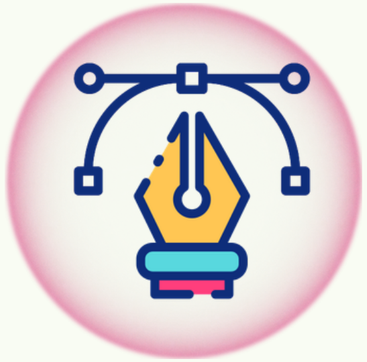
4 Recomendaciones



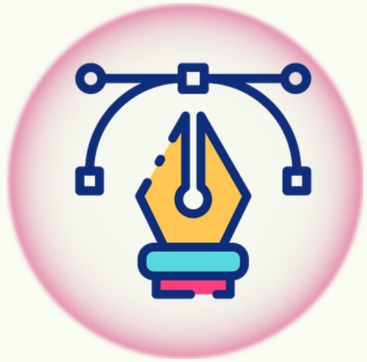
1.Reduzca la complejidad



2.Piense en los cambios que sufrirá el diseño



3.Reduzca el acoplamiento



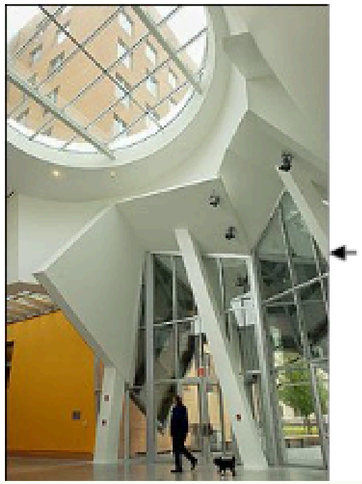
4.Prefiera la inyección de dependencias

1. Reduzca la complejidad

La elevada complejidad en un diseño es siempre una mala señal: si un diseño es complicado de entender y de explicar, posiblemente será difícil de implementar, mantener y expandir.

- Prefiera diseños simples sobre diseños complejos
  - Son más fáciles de construir y modificar, y suelen tener menos errores.
- Diseñe estructuras claras, en las cuales sea fácil identificar el rol de cada participante y sea claro encontrar dónde está cada cosa cuando se tenga que arreglar o cambiar.
- Procure que su diseño y su código sean *sostenibles*: ellos mismos revelan las decisiones que se tomaron para llegar a ese punto.

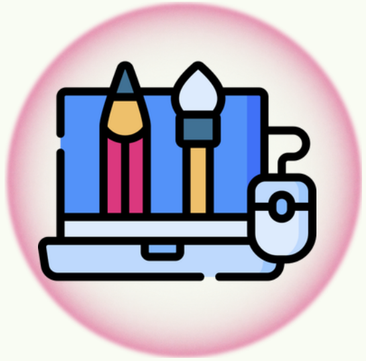




El Stata Center @ M.I.T. …

… tiene goteras

Piense en los cambios que sufrirá el diseño



Conocimiento:
TransferenciaBancaria probablemente tendrá que saber entre qué cuentas se hizo la transferencia, cuándo se hizo y cuál fue el monto.



Acciones:
ValidadorSaldo posiblemente tendrá entre sus responsabilidades validar el saldo de una cuenta.



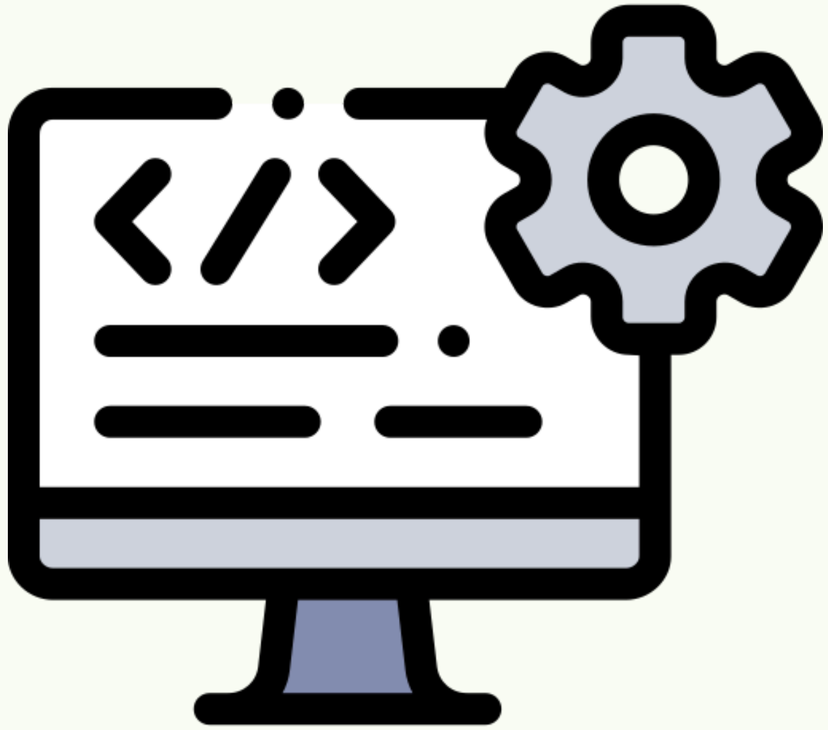
Decisiones:
SelectorFormato parece que tiene entre sus tareas tomar decisiones con respecto al formato de algo.

2.Piense en los cambios que sufrirá el diseño

Muchas aplicaciones sufrirán cambios después de que estén siendo utilizadas para corregir errores o para soportar nuevos requerimientos.



Aunque no es recomendable hacer diseños complicados para soportar requerimientos futuros que ni existen ni se conocen, sí tiene sentido diseñar considerando que en el futuro habrá cambios.



- Diseñe de forma tal que los cambios afecten partes pequeñas y predecibles del sistema.
- Cuando tenga sentido, diseñe el sistema para que sea modular.

Ejemplos de cambios para el caso de estudio

- Soportar peticiones HTTP-POST
- Soportar plantillas usando el formato de plantillas de Django
- Ofrecer un mecanismo de autenticación
- Permitir conexiones a varias Base de Datos
- Soportar bases de datos NoSQL
- Soportar enrutamiento reverso
- Mejorar el desempeño con un sistema de caché
- Soportar el despliegue en caliente de nuevas aplicaciones y páginas …

¿Recuerdan qué era el acoplamiento?

3.Reduzca el acoplamiento

El **acoplamiento** hace referencia a qué tantas y qué tan fuertes son las dependencias entre clases.

- Cuando el acoplamiento es **alto** entre dos clases, un cambio en una clase implica necesariamente cambios en la otra clase.
- Cuando el acoplamiento es **bajo** entre dos clases, cambios en una clase posiblemente no afecten a la otra clase.

3.Reduzca el acoplamiento

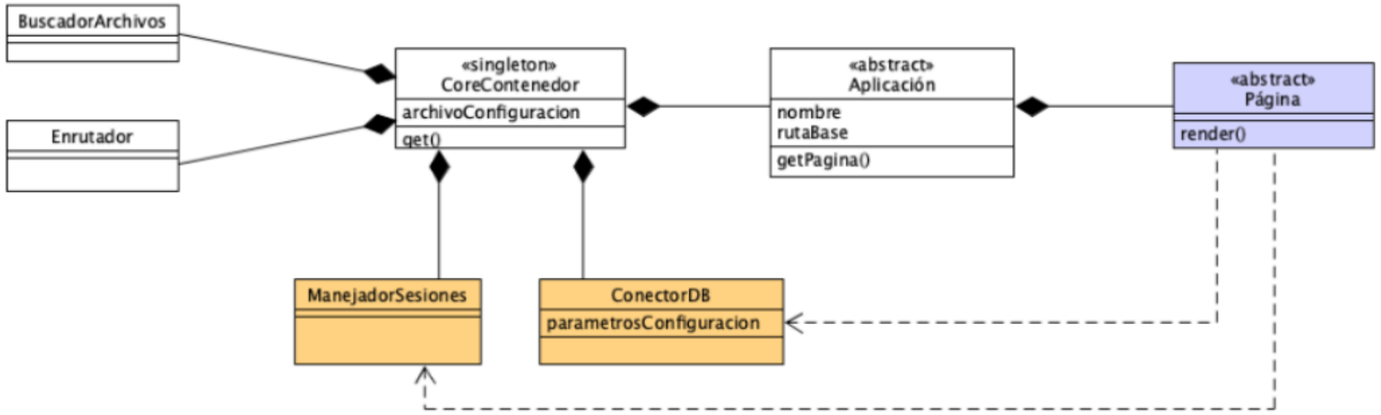
En general, es una buena idea procurar que el acoplamiento entre clases sea el menor posible, para que los cambios futuros en una clase no requieran tantos cambios en otras clases.

El **encapsulamiento** es fundamental para lograr un **bajo acoplamiento**.

Las dependencias entre clases deberían depender de las funcionalidades que ofrezca cada clase y no de cómo estén implementadas.



4.Prefiera la inyección de dependencias



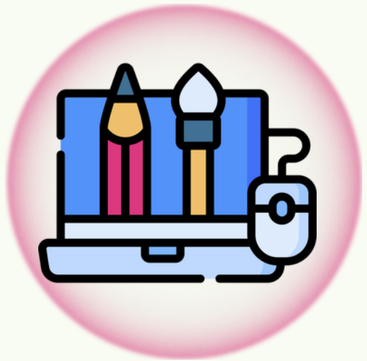
En este caso, sabemos que la clase Pagina tiene dependencias hacia ManejadorSesiones y ConectorDB, pero ¿cómo una instancia de Pagina va a tener acceso a instancias de estas dos clases?

- **Alternativa 1**: buscándolas, a través de la aplicación y de CoreContenedor.
- **Alternativa 2**: dejando que CoreContenedor le inyecte esas dependencias Pagina.

La segunda estrategia es la que se conoce como inyección de dependencias y es muy utilizada para reducir el acoplamiento.

Lo anterior, es algo que hacen ya

Inyectar dependencias



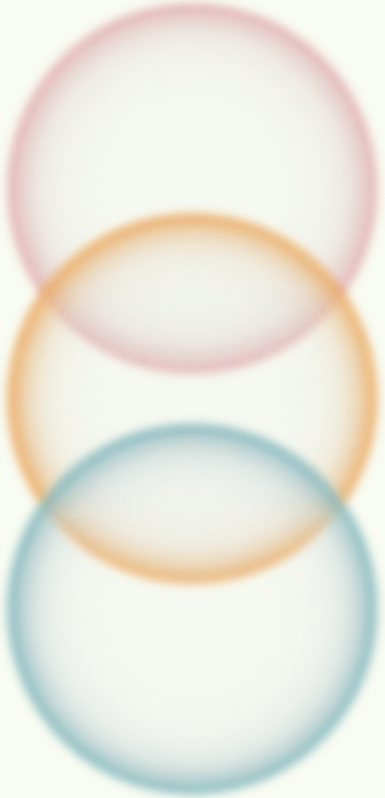
Por método constructor



Por métodos setter

```
public class ServicioDeFacturas {
    private readonly IRepositorioDeFacturas _repositorio;

    // La dependencia se inyecta a través del constructor
    public ServicioDeFacturas(IRepositorioDeFacturas repositorio) {
        _repositorio = repositorio;
    }
    // ... resto del servicio ...
}
```



Ahora, miremos unos consejos cuando asignan responsabilidades

Asignación progresiva de responsabilidades

1. Empiece asignando **las responsabilidades más gruesas**: las que son más complejas, requieren más información y tendrán un impacto más grande definiendo la estructura global del sistema.

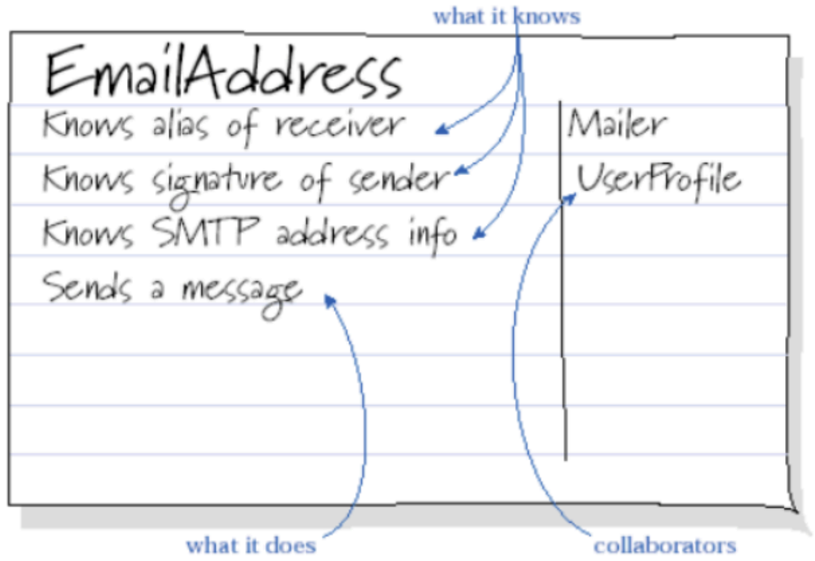
2. Sea **oportunista**: si ve que hay una responsabilidad que ya sabe quién debería asumir, asígnesela.

3. No hay una solución óptima y el proceso es **iterativo**. A veces hay que arriesgarse y revisar más adelante si la decisión fue correcta o no.

Descripción de las responsabilidades

1. Escriba descripciones para las responsabilidades que sean precisas y no usen verbos vagos o débiles (ej. manejar, gestionar).
   - Asignar responsabilidades muy vagas es difícil (y peligroso).

2. No exagere con las descripciones de las responsabilidades: precisión no significa que tiene que haber párrafos enteros para cada descripción. Una frase corta debería ser suficiente.



Concentrar vs Distribuir

Hay que buscar el balance entre concentrar responsabilidades y repartirlas entre varios elementos especialmente cuando las responsabilidades son complejas

- Evite que algún elemento tenga demasiadas responsabilidades.
- Haga que los elementos sean coherentes. Algo es coherente si las cosas que hace están relacionadas entre ellas.
- Intente restringir las responsabilidades de un objeto a un solo dominio.



Ajuste las responsabilidades

1. Si cuando vaya a hacer la asignación una responsabilidad le parece muy grande o le parece que puede ajustarse a muchos elementos, descompóngala en responsabilidades más pequeñas que pueda distribuir mejor.

2. No repita ni sobreponga responsabilidades: si hay más de un elemento capaz de hacer lo mismo, eventualmente tendrá problemas cuando tenga que hacer cambios.

    - Las clases marcadas en Java como **`deprecated`** son un ejemplo de esto: hay clases que hace más de 15 años se marcaron de esta manera para indicar que alguien más había asumido sus responsabilidades, y aún así siguen existiendo dentro del JDK porque sigue habiendo dependencias hacia ellas.

Ahora, miremos de nuevo los estilos de control

Estilos de control



- Si el estilo de control es **centralizado**, la toma de decisiones se concentra en pocos elementos (típicamente, **controladores**).
- Si el estilo de control es **delegado**, la toma de decisiones se divide entre algunos elementos centrales y otros elementos más pequeños. En general los primeros terminan actuando como **coordinadores**.
- Si el estilo de control es **disperso**, la toma de decisiones se distribuye entre muchos elementos. En este caso las decisiones tienden a ser más pequeños y se vuelve difícil ubicar los centros de control de la aplicación.

Centralizado

- La responsabilidad de tomar decisiones está concentrada en pocos elementos
  - Estos elementos típicamente son Controladores

- La lógica de control tiende a volverse compleja dentro de los controladores porque se concentra en ellos
- Los controladores tienden a volverse dependientes de los information holders que mantienen toda la información que ellos necesitan para hacer sus tareas y tomar decisiones
- Todo el trabajo interesante dentro de la aplicación ocurre dentro de los controladores; los otros elementos terminan siendo muy sencillos y poco interesantes

✓ Es fácil encontrar los puntos críticos dentro del código, en los cuales se toman decisiones y se implementan algoritmos interesantes

Delegado

- El control se distribuye no sólo en los controladores sino también en otros objetos de la solución
  - Estos elementos típicamente serán Coordinadores

✓ Los coordinadores necesitan saber menos sobre otros objetos (menor acoplamiento) y sólo les interesan las interfaces
✓ Los diálogos entre elementos son de un nivel más alto: en lugar de muchos mensajes pidiendo información puntual y específica, se usan mensajes solicitando la ejecución de tareas más interesantes.
✓ Los cambios, por ejemplo en algún algoritmo, afectan a menos elementos.
✓ Es más fácil distribuir el trabajo entre varios miembros de un equipo de desarrollo.

○ Puede haber colaboraciones muy complejas
○ Puede haber muchas colaboraciones sin que haya mucho trabajo real

Disperso

- La toma de decisiones está repartida entre todos los elementos involucrados en una tarea: no es claro que haya un coordinador o controlador.

- Cada uno de los participantes tiene responsabilidades pequeñas, cada decisión que se tome es pequeña, y para tomar decisiones más grandes es necesario agrupar el resultado del trabajo de muchos elementos.
- Es difícil encontrar los centros de control para poder corregir errores o introducir mejoras y cambios.
- Cualquier cambio en el sistema puede afectar a muchos elementos.

Último consejo

Características de un documento de diseño

- Puede ser informal: debe ser el documento adecuado para el proyecto
- No es un manual de implementación para la solución

- Contenido típico
  - Contexto y alcance, para conocer las restricciones bajo las cuales se diseñe la solución
  - Objetivos y no-objetivos
    - Los no-objetivos son las cosas que explícitamente se quieren dejar por fuera de la solución
  - El diseño, presentando tanto los elementos que hacen parte de la solución como la justificación para las decisiones más complejas que se hayan tenido que tomar
    - Elementos de la solución
    - Diagramas de contexto del sistema
    - APIs
    - Información sobre la persistencia
    - Código o Pseudo-código para los algoritmos críticos
  - Alternativas que se hayan considerado
  - Preocupaciones transversales que se deban considerar (ej, seguridad, transaccionalidad, etc.)